
Propuesta de plataforma de despliegue de un proyecto de inteligencia artificial basado en el desarrollo experimental de un clasificador de mensajes de odio

Investigación y Gestión de Proyectos en Inteligencia Artificial

Luis Miguel Garay, Paula Coll Lapido

Asier Marin Fernandez

31 de enero de 2025

Resumen

Este estudio presenta una plataforma de despliegue para un clasificador de mensajes de odio basado en inteligencia artificial, utilizando metodologías ágiles y MLOps. Se comparan distintas estrategias de desarrollo de software, destacando la transición de modelos tradicionales a enfoques más dinámicos como Scrum, Kanban y DevOps. Se define un ciclo de vida del modelo estructurado en etapas clave como la preparación de datos, entrenamiento, evaluación y monitoreo. La propuesta de despliegue se basa en tecnologías como Kubernetes, KubeFlow y KServe, descartando soluciones MLaaS por la necesidad de una infraestructura más escalable. Finalmente, se incluyen aspectos de seguridad y una estimación de costes.

I. Introducción

Con la mejora de la tecnología y la aparición de nuevas herramientas de comunicación, hace casi dos décadas empezaron a proliferar plataformas de redes sociales. Las redes sociales siempre han protegido la privacidad de los usuarios en el anonimato con el uso de apodos. Gracias a esta privacidad no es posible saber con exactitud qué persona se esconde detrás de cada cuenta y ha permitido expresiones que en la vida diaria no ocurren. Mensajes mal sonantes, insultos, amenazas, etc. Estos mensajes son conocidos como mensajes de odio.

Con Inteligencia Artificial¹ se pueden desarrollar herramientas para detectar y proteger a los usuarios de estos mensajes. Para poder abordar esta necesidad se debe identificar de qué tipo de proyecto se trata desde una perspectiva científica. La clasificación de odio se podría hacer mediante una clasificación binaria simple, es decir, el mensaje contiene odio o no. Esta clasificación puede ser resuelta mediante técnicas de Aprendizaje Automático Supervisado². Desarrollar cualquier proyecto de IA enfrenta numerosos desafíos, como diseñar la forma de trabajar y metodologías. Definir un ciclo de vida del proyecto en base a sus criterios empresariales y técnicos, además de métodos de desarrollo modernos y eficientes.

II. Metodología

Los proyectos de IA tienen ciertas similitudes respecto a otros proyectos de desarrollo de software tradicionales. Las metodologías de desarrollo han ido evolucionando hacia una perspectiva más ágil. Antiguamente, los proyectos de software estaban orientados a arquitecturas más monolíticas y los proyectos seguían una línea de desarrollo en cascado. Este tipo de desarrollo supone varios inconvenientes, como

¹Disciplina científica que se ocupa de crear programas informáticos que ejecutan operaciones comparables a las que realiza la mente humana.

²Disciplina de IA que permite que un sistema aprenda y mejore de forma autónoma.

la falta de flexibilidad y la dificultad de adaptación a cambios. Es común que los proyectos de software estén en constante cambio respecto a otras metodologías más tradicionales. Tradicionalmente ha existido diferentes metodologías de desarrollo:

- I) Modelo en cascada: Flujo continuo de desarrollo en la que una fase no comienza hasta que la anterior ha finalizado.
- II) Modelo interactivo e incremental: Basado en tareas que se agrupan en pequeñas interacciones.
- III) Modelo unificado en proceso: Conjunto de fases ligeramente más flexibles que el modelo en cascada. Se basa en fases incrementales.
- IV) Metodología ágil *Scrum*: Basada en la adaptación a cambios y en la entrega de software funcional en periodos cortos. Se realizan reuniones diarias y se analizan el estado de las tareas.
- V) Metodología ágil *Kanban*: Se basa en la ordenación de tareas en un tablero que simboliza las fases. Se puede usar conjuntamente con *Scrum*.

La industria del desarrollo del software ha adquirido una tendencia ascendente hacia las metodologías ágiles. Esto se debe a la necesidad de la implementación de cambios constantes de manera rápida.

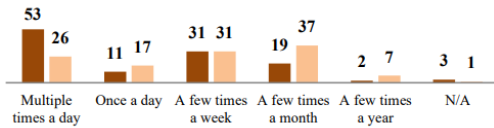


Figura 1: Encuesta sobre el número de despliegues por día.

Nota: Adoptado de *Beyond Continuous Delivery: An Empirical Investigation of Continuous Deployment Challenges* [Diagrama], por (Shahin et al., 2017),

Fuente

(<https://ieeexplore.ieee.org/document/8170091>).

Dentro del abanico de las metodologías de desarrollo ágil, también se encuentra el concepto de *DevOps* que está enfocado en el desarrollo de *software*. *DevOps* es un paradigma de la metodología que impulsa la mejora del desarrollo de manera rápida. El concepto de agilidad se fusiona con la mejora y entrega continua de cambios. Esta filosofía tradicional, también se puede enfocar más concretamente en el desarrollo de *software* de IA con el concepto de *MLOps*. *MLOps* es una metodología que se puede ajustar perfectamente a proyectos de aprendizaje automático con un ciclo de vida diseñado específicamente para este ámbito.

DevOps y *MLOps* se diferencian en ciertos aspectos. En los productos de software tradicionales no es necesario la monitorización de un modelo y está centrado en la entrega continua de nuevas funcionalidades. *DevOps* también se define con un ciclo de vida más corto y sencillo, en la que no existe entrenamiento ni validación de modelos.

III. Modelos de clasificación

Existe una variedad de modelos que se pueden entrenar para hacer predicción de regresión y clasificación. El caso de entrenamiento de modelos de clasificación, el objetivo es asignar una etiqueta preestablecida. Para el caso de mensajes de odio, la etiqueta será 'odio' y 'no odio'.

El entrenamiento de un clasificador de odio debe contar con una serie de especificaciones. El modelo admitirá datos categóricos. Además se considerará su complejidad espacial y temporal. Teniendo en cuenta que se puede implementar en redes sociales, la escalabilidad debe ser algo a tener en cuenta. Por lo tanto, podrá admitir gran cantidad de datos.

Las predicciones de clase producen un valor de predicción en forma numérica continua que se refiere a una probabilidad de pertenencia. Existen numerosos modelos que pueden servir para regresión y clasificación, se puede hacer una comparación de los modelos más relevantes para una clasificación binaria sobre categorías discretas.

- I) Árbol de decisión: Divide los datos en ramas basadas en reglas de decisión.
- II) Máquinas de Vectores de Soporte (SVM): Encuentra el hiperplano óptimo que separa las clases con máximo margen.
- III) Clasificador Naïve Bayes: Usa probabilidades para clasificar con base en el Teorema de Bayes y asume independencia entre atributos.

Para la elección del modelo se tendrá en cuenta las siguientes métricas:

- I) Soporte de datos numéricos.
- II) Soporte de datos categóricos.
- III) Complejidad temporal del modelo, si el entrenamiento es un proceso rápido.
- IV) Complejidad espacial, si es un modelo que es viable con datos grandes.

Modelo	D. Num.	D. Cat.	C.T.	C.E.
DT	Admite	Admite	Lento	No
SVM	Admite	Complejo	Muy lento	No
NBC	Discret.	Admite	Rápido	Sí

Cuadro 1: Tabla comparativa modelos DT, SVM y NBC para datos números, datos categóricos, complejidad temporal y complejidad espacial (escalabilidad)

En base a las especificaciones que se han definido, el modelo Naïve Bayes puede ser una mejor opción. Este modelo admite datos categóricos, algo esencial para ser entrenados con texto de las redes sociales. El modelo también es más rápido en entrenamiento que los competidores y además tiene capacidad de escalabilidad.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \quad (1)$$

IV. Ciclo de vida

El ciclo de vida de un proyecto de IA debe ser flexible y abierto a cambios constantes. Una vez que el modelo se ha desplegado en producción, el rendimiento se degrada respecto a la frecuencia de los cambios de los datos (Garg et al., 2021).

MLOps se puede dividir en 3 diferentes niveles; *MLOps 0*, *MLOps 1* y *MLOps 2*. La diferencia principal entre el nivel 0 y el nivel 1 es el nivel de orquestación y automatización de entrenamiento, mientras que, en el último nivel existe una automatización completa (Garg et al., 2021).

Se pueden definir las etapas del ciclo de vida del proyecto como la siguiente:

- i) Preparación de datos: Limpieza de datos y segmentación de los datos de entrenamiento y validación. En este caso se descartan datos que contengan palabras o ‘tokens’ inválidos. También se realiza una separación de los datos de entrenamiento y prueba.
- ii) Selección del algoritmo de clasificación. Se adapta el modelo de Naïve Bayes mencionado anteriormente.
- iii) Selección y modificación de parámetros del entorno o hiperparámetros.
- iv) Entrenamiento y ajuste del modelo.
- v) Evaluación del modelo.
- vi) Monitorización.

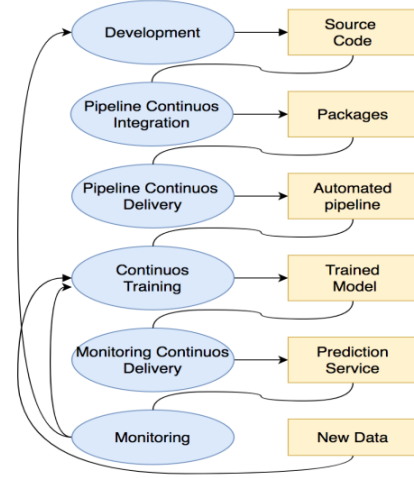


Figura 2: Ejemplo de ciclo de vida *MLOps*.

Nota: Adoptado de *On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps* [Diagrama], por (Garg et al., 2021), Fuente (<https://ieeexplore.ieee.org/document/9723793>).

Las últimas etapas no se deben percibir como un ciclo cerrado sin retorno, sino como un ciclo de mejora continua. Existen diferentes mecanismos para evaluar el rendimiento de un modelo, y en base a esos resultados se puede ajustar los diferentes parámetros del modelo. Se define una matriz de confusión que mide los verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN).

IV.I. Exactitud

La exactitud mide el porcentaje de predicciones correctas.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

IV.II. Precisión

La precisión mide cuántos de los casos predichos de manera positiva realmente lo son:

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

IV.III. Recall

La exhaustividad mide cuántos de los casos positivos fueron detectados correctamente:

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

IV.IV. Puntuación F1

El F1-score es el promedio armónico entre precisión y recall:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (5)$$

IV.V. Curva ROC y AUC

La curva ROC muestra la relación entre la Tasa de Verdaderos Positivos (TPR) y la Tasa de Falsos Positivos (FPR).

$$TPR = \frac{TP}{TP + FN} \quad (6)$$

$$FPR = \frac{FP}{FP + TN} \quad (7)$$

V. Plataforma de despliegue

Una plataforma de despliegue basado en metodologías ágiles y *MLOps*, debe abarcar la integración, entrega y despliegue continuo. Existen diversas plataformas de aprendizaje automático como servicio, también denominadas *MLaaS*. Estas plataformas permiten el despliegue de manera sencilla y que puede ser una buena opción para proyectos sencillos. En este caso, para la clasificación de mensajes de odio, se ha determinado que será necesario de una infraestructura escalable y personalizada. Por este motivo, plataformas como *Google Cloud Platform*, *Amazon SageMaker* o *Microsoft Azure ML* están descartadas como una opción viable.

Atrás quedó la época en la que un abonado de computación en la nube tenía que administrar sus propios servidores o elementos físicos de *hardware*. Estos mecanismos de administración tradicionales eran lentos y con poca capacidad de escalabilidad. La industria ha evolucionado a modelos de virtualización con componentes más ligeros basados en contenedores. El uso de virtualización mediante *Hypervisor* es un proceso que a día de hoy sigue siendo utilizado para tareas concretas, pero no es aconsejable para servidores con necesidad de escalabilidad por inflexibilidad y mayor consumo de recursos adicionales.

Los contenedores son rápidos y ligeros ya que no necesitan su propio sistema operativo completo. Es fácilmente escalable y portable, una opción perfecta para metodologías *DevOps* y *MLOps*.

En consecuencia, se ha decidido desarrollar una plataforma de despliegue basado en contenedores y orquestación. *Kubernetes* es la plataforma de orquestación de contenedores más usado en el mercado de la

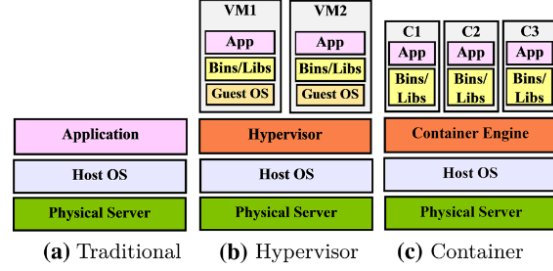


Figura 3: Comparativa de arquitectura de virtualización tradicional, *Hypervisor* y contenedor.

Nota: Adoptado de *Virtualization in Cloud Computing: Moving from Hypervisor to Containerization—A Survey* [Diagrama], por (Bhardwaj y Krishna, 2021), Fuente (<https://link.springer.com/article/10.1007/s13369-021-05553-3>).

computación en la nube. Existen además, plataformas que adaptan los proyectos de IA en este orquestador.

Se propone una plataforma de despliegue con un proceso de CI/CD abstraído en procesos de máquinas virtuales o *pipelines*. El proceso de construcción y empaquetado del código se ejecutaría en este proceso. Posteriormente, se produciría un flujo basado en *Kubeflow*. *Kubeflow* es una plataforma de código abierto que se implementa sobre *Kubernetes*. Integra la parte *MLOps*, necesaria para cualquier proyecto de IA y su responsabilidad es el procesamiento de datos, entrenamiento y evaluación. En la siguiente figura se muestra un ejemplo de *MLOps* integrado en el proceso tradicional CI/CD.

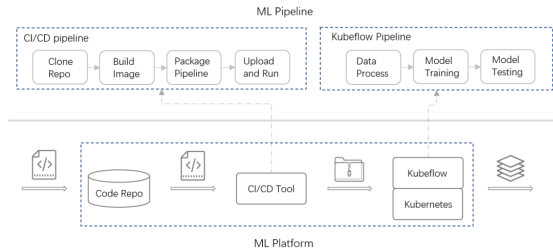


Figura 4: Ejemplo de plataforma de despliegue *MLOps*.

Nota: Adoptado de *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)* [Diagrama], por (Zhou et al., 2020), Fuente (<https://ieeexplore.ieee.org/document/9361315>).

Para el despliegue del modelo se propone *KServe*. *KServe* es una plataforma de despliegue que permite la inferencia en *Kubernetes*. Además proporciona la abstracción para marcos de trabajo con TensorFlow,

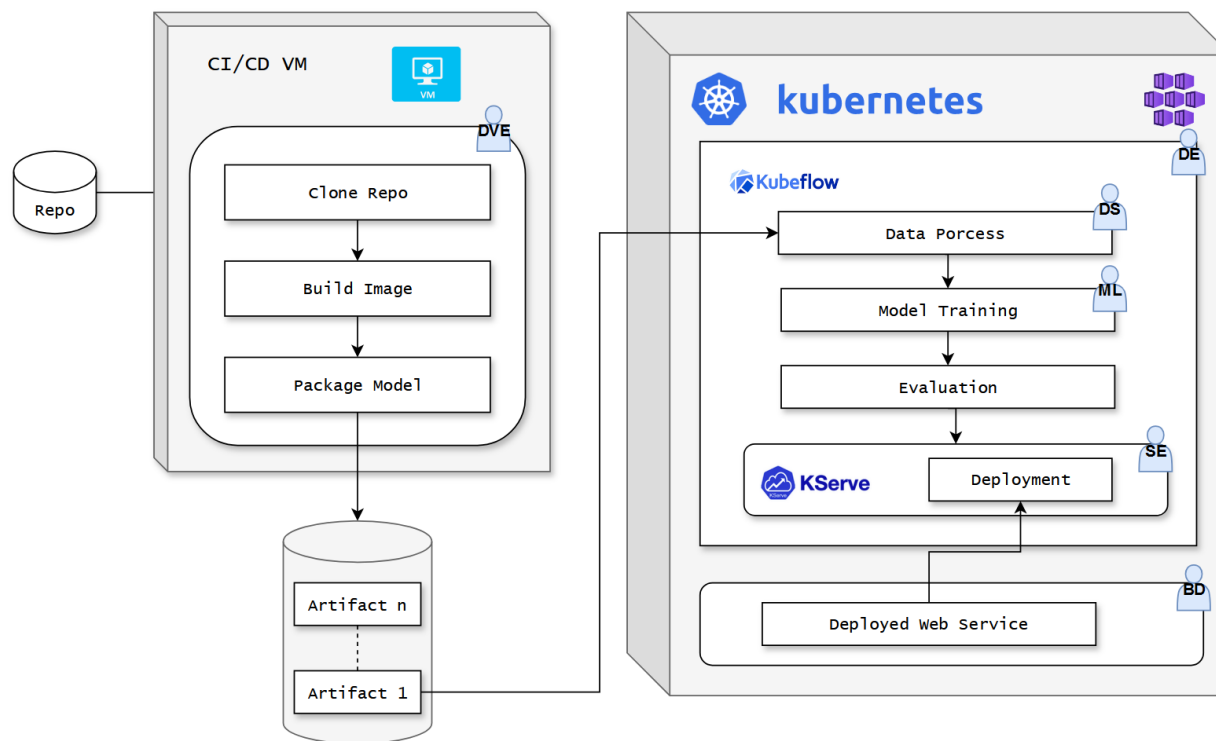


Figura 5: Diseño plataforma de despliegue completa. Se puede ver el flujo completo de un proceso de despliegue. Se lleva a cabo un proceso de integración y despliegue continuo donde se despliega la aplicación. También se puede observar la integración de los distintos perfiles de ingeniería.

XGBoost, scikit-learn, PyTorch y ONNX para resolver casos de uso de servicio de modelos de producción.

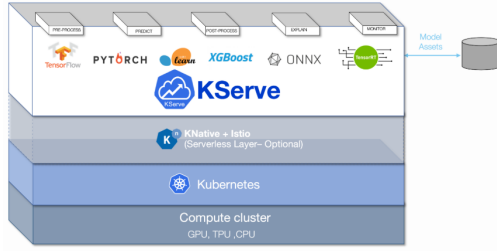


Figura 6: Arquitectura *KServe*.

Nota: Adoptado de *KServe: The next generation of KFServing* [Diagrama], por («KServe», 2021), Fuente (<https://blog.kubeflow.org>).

VI. Roles

- I) Data Scientist (DS): Responsable de la preparación de los datos y entrenamiento del modelo (Amrit y Narayanappa, 2025).
- II) ML Engineer (ML): Responsable de la integración de los modelos en la plataforma de despliegue (Amrit y Narayanappa, 2025).
- III) Backend Developer (BD): Responsable de la integración de la plataforma de despliegue con la aplicación (Amrit y Narayanappa, 2025).
- IV) Data Engineer (DE): Responsable de la integración de los datos en la plataforma de despliegue (Amrit y Narayanappa, 2025).
- v) Devops Engineer (DEV): Responsable de la integración de la plataforma de despliegue con el proceso de CI/CD (Amrit y Narayanappa, 2025).
- VI) Software engineer (SE): Responsable de la integración de la aplicación con la plataforma de despliegue (Amrit y Narayanappa, 2025).

VII. Mecanismos de seguridad

Kubernetes dispone de mecanismos de seguridad, proporcionando autenticación y autorización.

La API Server controla el acceso y requiere autenticación. Métodos comunes incluyen:

- Autenticación mediante Certificados X.509
- Tokens de Servicio (Service Accounts)
- Estándar OAuth y OIDC

■ Webhook Authentication

Ejemplo de autenticación con una cuenta de servicio:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: mi-cuenta-de-servicio
```

Los métodos principales de autorización incluyen:

- RBAC (Role-Based Access Control)
- ABAC (Attribute-Based Access Control)
- Webhook Authorization

Ejemplo de RBAC para acceso de solo lectura a pods:

```
apiVersion: rbac.authorization
kind: Role
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

También se pueden implementar políticas de restricción de comunicación entre unidades de trabajo. *Kubernetes* también ofrece cifrar variables de entorno sensibles con objetos *Secrets*.

VIII. Estimación de costes

Costes mensuales adjuntados como anexo en un archivo de Excel. Estimación mensual en la plataforma de *Microsoft Azure*.

Referencias

- Amrit, C., & Narayanappa, A. K. (2025). An analysis of the challenges in the adoption of MLOps. *Journal of Innovation Knowledge*, 10(1), 100637. <https://doi.org/https://doi.org/10.1016/j.jik.2024.100637>
- Bhardwaj, A., & Krishna, C. R. (2021). Virtualization in cloud computing: Moving from hypervisor to containerization—a survey. *Arabian Journal for Science and Engineering*, 46(9), 8585-8601.

- Garg, S., Pundir, P., Rathee, G., Gupta, P., Garg, S., & Ahlawat, S. (2021). On Continuous Integration / Continuous Delivery for Automated Deployment of Machine Learning Models using MLOps. *2021 IEEE Fourth International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, 25-28. <https://doi.org/10.1109/AIKE52691.2021.00010>
- KServe: The next generation of KFServing. (2021, septiembre). Consultado el 23 de enero de 2025, desde <https://blog.kubeflow.org/release/official/2021/09/27/kfserving-transition.html>
- Shahin, M., Babar, M. A., Zahedi, M., & Zhu, L. (2017). Beyond Continuous Delivery: An Empirical Investigation of Continuous Deployment Challenges. *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 111-120. <https://doi.org/10.1109/ESEM.2017.18>
- Zhou, Y., Yu, Y., & Ding, B. (2020). Towards MLOps: A Case Study of ML Pipeline Platform. *2020 International Conference on Artificial Intelligence and Computer Engineering (ICAICE)*, 494-500. <https://doi.org/10.1109/ICAICE51518.2020.00102>